

Power BI 2027 Prep

Agentic, AI-ready development. TMDL, UDFs, PBIP+Git, MCP, Copilot, Fabric IQ — in the order you should learn them.

A 7-module tutorial course

Power BI 2027 Prep: Agentic, AI-Ready Development

A preparation tutorial for Power BI developers who want to stay employable through 2027. The industry is moving from "build reports" to "engineer AI-ready analytics solutions," and the skill stack is widening fast. This course walks through what to learn, in what order, and how to actually practice each piece.

Reality check up front. You cannot learn most of this by reading. TMDL view, PBIP, Visual Calculations, MCP, Copilot — all require Power BI Desktop (Windows) and, for several topics, a Fabric or Power BI Premium / PPU workspace. This doc gives you the mental model, the setup checklist, the commands/code, and the exercises. Follow it with hands-on practice or it won't stick.

Who this is for

- Power BI developers who shipped reports in 2023–2025 and now feel the ground shifting under them.
- Analytics engineers who use Power BI as a presentation layer on top of dbt / Fabric / Snowflake and want to speak the modern Power BI dialect.
- Data scientists who need to hand models to a BI team and want to understand what "semantic model" and "agentic authoring" actually mean.

The learning order

Learn the four foundational skills from the infographic **in order**. Each one makes the next easier.

#	Skill	Why it's first	Module
1	TMDL view	Your whole semantic model as plain text. Pre-req for Git, code review, and anything agentic.	01-tmdl-view.md
2	User-defined functions (UDFs)	Write a DAX calc once, reuse everywhere. Shrinks model size and makes AI-generated DAX reviewable.	02-udfs.md
3	PBIP + Git	File-based project format. Diffs, pull requests, CI/CD — the software-engineering practices Power BI was missing.	03-pbip-git.md
4	MCP + AI agents	Tell an agent what you need; it edits the model for you. Only safe once you can read TMDL and review Git diffs.	04-mcp-ai-agents.md

Then widen into the ecosystem:

#	Topic	Module
5	Copilot + Fabric IQ	05-copilot-fabric-iq.md
6	Service features: DirectQuery semantic models, Visual Calculations, Design ribbon, Outlook live visuals, Initial axis scroll	06-service-and-ux.md
7	The 2027 skill stack and a 90-day roadmap	07-roadmap-and-skill-stack.md

What's GA vs Preview (as of late 2025 / 2026)

GA = generally available, safe for production. Preview = usable but can change; don't bet a client deliverable on it.

Feature	Status
Power BI Projects (PBIP) + TMDL	GA
Visual Calculations	GA
Initial Axis Scroll Position	GA
New Design Ribbon	GA
Power BI Agentic Experiences (Authoring, Desktop Bridge)	GA
User-defined functions (DAX UDFs)	GA
Microsoft 365 Copilot + Power BI	Preview
Fabric IQ + Power BI Copilot	Rolling GA
Remote Power BI Authoring MCP	Preview
DirectQuery semantic models in Service	GA
Live Power BI visuals in Outlook	Preview

Always double-check the Power BI release wave notes before telling a client "this is GA" — Microsoft shifts status between monthly releases.

Setup you need before Module 1

1. **Windows machine or VM** (Power BI Desktop is Windows-only; Mac users: Parallels, a cloud Windows VM, or Fabric web authoring).
2. **Power BI Desktop** — current month's release. Download: <https://aka.ms/pbidesktopstore>.
3. **A Fabric or Power BI PPU workspace** for the Service-side features (Modules 5, 6). A free Fabric trial works.
4. **Git** — any client. Command line is enough. `git --version` should return something.
5. **VS Code** with the "Power BI Projects" extension (search `pbip`). You will edit TMDL in VS Code more than in Desktop.
6. **An AI coding assistant that speaks MCP** — Claude Code, Cursor, or Copilot in VS Code. Needed for Module 4.

Nice-to-have:

- **Tabular Editor 3** (paid) or **Tabular Editor 2** (free) for advanced TMDL and best-practice analyzer rules.
- **DAX Studio** for query plans and performance tuning.

How to use this course

Each module is self-contained and follows the same shape:

1. **What it is** — one paragraph, no jargon.
2. **Why it matters now** — the problem it solves.
3. **The mental model** — the one picture or table that unlocks the topic.
4. **Hands-on walkthrough** — step-by-step, with commands and code.
5. **Exercises** — do these. Don't skip.
6. **Common traps** — mistakes I've seen cost people days.
7. **Further reading** — Microsoft Learn + community links.

Pace yourself: one module per weekend is reasonable. Rushing through TMDL to get to the "fun" MCP module is the fastest way to deploy an agent that quietly breaks your semantic model.

The bigger picture

Power BI + Fabric + Copilot + MCP + PBIP + TMDL + Git + AI Agents

is the stack. The semantic model is the bridge between enterprise data and AI. If you own that bridge, you are valuable through 2027 and beyond. If you only know the ribbon in Desktop, you are not.

Start with [Module 1: TMDL view](#).

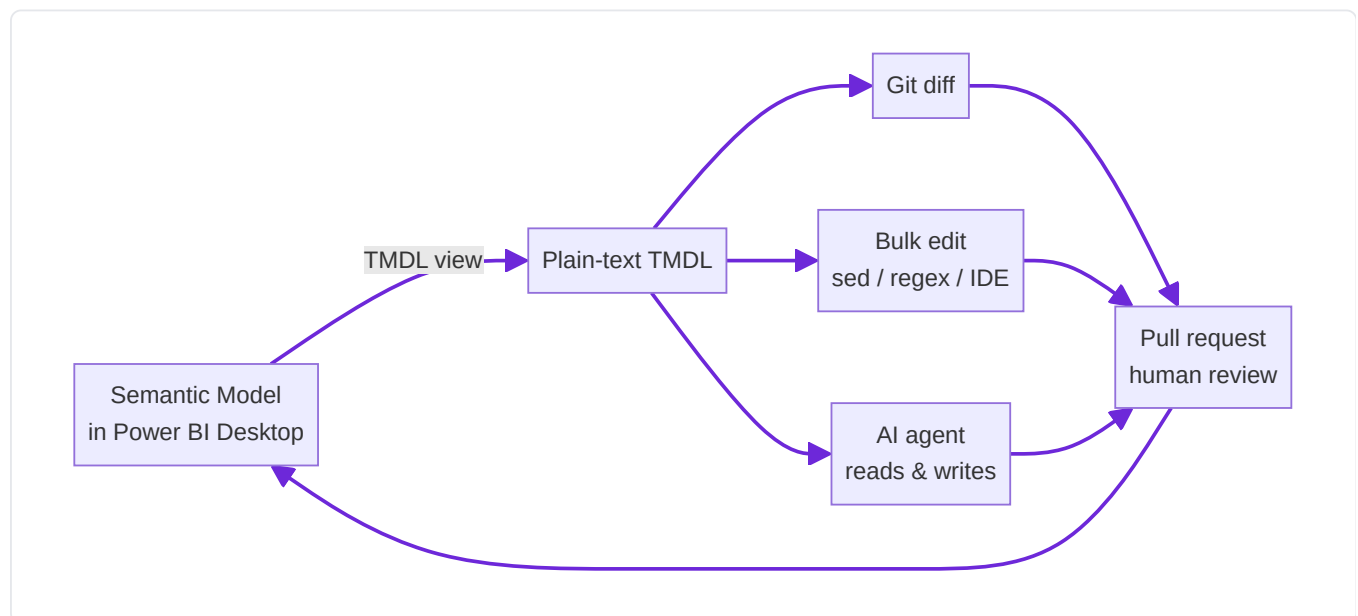
Module 1 — TMDL View

TMDL = Tabular Model Definition Language. It's your whole semantic model expressed as plain text, with Python/YAML-like indentation. Open TMDL view inside Power BI Desktop and you can edit tables, columns, measures and relationships as code — no clicking through a GUI.

Why it matters now

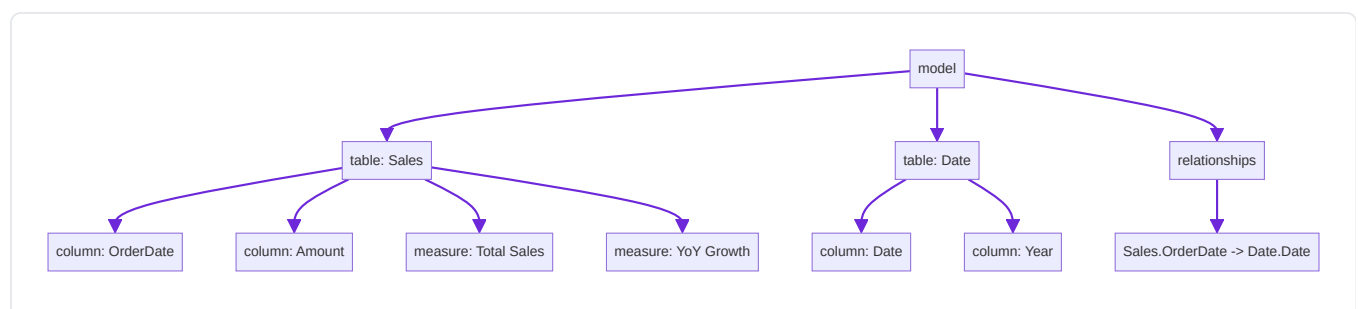
Before TMDL, a Power BI model was a sealed binary blob (`.pbix`). You couldn't diff it, you couldn't bulk-edit, you couldn't review it in a pull request, and you couldn't let an AI touch it safely. TMDL fixes all four. Every later module in this course assumes you are comfortable reading and editing TMDL.

The mental model



The core move: a round-trip between the Desktop UI and the text. You author in whichever is faster — the UI for one measure, the text for twenty.

The TMDL object hierarchy



Each level maps to a block of indented TMDL. Learning the hierarchy is most of learning TMDL.

Hands-on walkthrough

1. Open TMDL view

In Power BI Desktop → **View** ribbon → **Model view** → the TMDL button (looks like `</>`). You now see a text pane next to the diagram.

2. Read a measure

Click any measure in the model. The TMDL pane shows:

```
measure 'Total Sales' = SUM ( Sales[Amount] )
  formatString: "$#,##0"
  displayFolder: KPIs
```

Indentation matters. One tab (or four spaces) consistently.

3. Add a measure as text

Type a new measure directly:

```
measure 'Total Sales LY' =
  CALCULATE (
    [Total Sales],
    SAMEPERIODLASTYEAR ( 'Date'[Date] )
  )
  formatString: "$#,##0"
  displayFolder: KPIs
  description: "Prior-year equivalent of Total Sales."
```

Hit **Apply changes**. The measure appears in the model instantly.

4. Bulk-rename a display folder

Suppose you have 40 measures in `displayFolder: KPIs` and you want `displayFolder: 01 KPIs` (so it sorts first).

- Click the **table** header in the TMDL pane (not a single measure) to get the whole table's TMDL.
- Find/replace `displayFolder: KPIs` → `displayFolder: 01 KPIs`.
- Apply. All 40 measures move. Try that with the GUI.

5. See where TMDL lives on disk

Save the file as a **PBIP** (Module 3) and look at the folder:

```
MyReport.SemanticModel/  
├─ definition/  
│   └─ tables/  
│       ├── Sales.tmdl  
│       └─ Date.tmdl  
│   └─ relationships.tmdl  
│   └─ model.tmdl  
│   └─ database.tmdl  
└─ definition.pbism
```

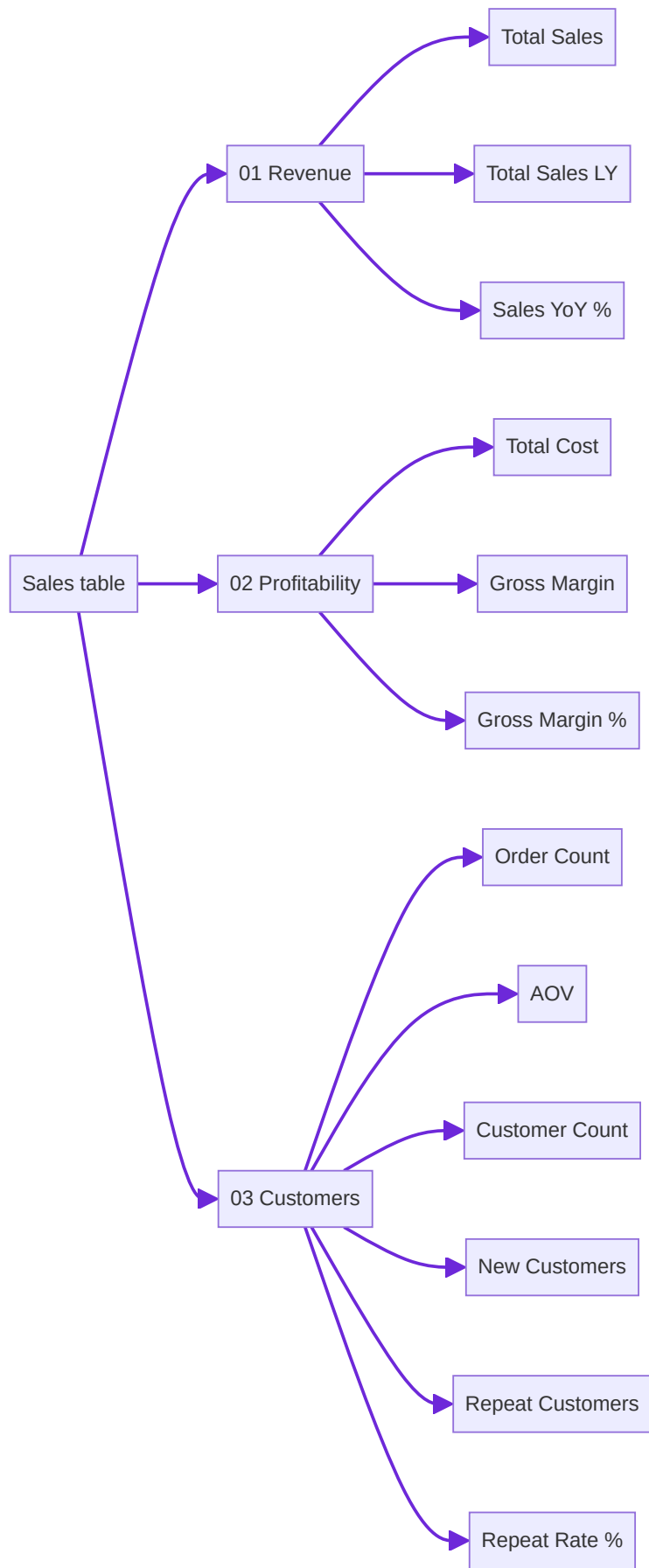
Every table is its own file. This is why Git can diff a semantic model.

A worked example: refactor measures into display folders

Starting state — a flat list of 12 measures, no folders:

```
measure 'Total Sales' = SUM ( Sales[Amount] )  
measure 'Total Sales LY' = CALCULATE ( [Total Sales], SAMEPERIODLASTYEAR ( 'Date'[Date] ) )  
measure 'Sales YoY %' = DIVIDE ( [Total Sales] - [Total Sales LY], [Total Sales LY] )  
measure 'Total Cost' = SUM ( Sales[Cost] )  
measure 'Gross Margin' = [Total Sales] - [Total Cost]  
measure 'Gross Margin %' = DIVIDE ( [Gross Margin], [Total Sales] )  
measure 'Order Count' = DISTINCTCOUNT ( Sales[OrderID] )  
measure 'AOV' = DIVIDE ( [Total Sales], [Order Count] )  
measure 'Customer Count' = DISTINCTCOUNT ( Sales[CustomerID] )  
measure 'New Customers' = ...  
measure 'Repeat Customers' = ...  
measure 'Repeat Rate %' = DIVIDE ( [Repeat Customers], [Customer Count] )
```

Target state — three folders:



In TMDL, append `displayFolder:` to each measure. You can do this with a multi-cursor edit in VS Code in about 30 seconds. That edit, in the GUI, is 12 right-clicks.

Exercises

1. Open an existing `.pbix` you own. Save as PBIP. Open TMDL view. Rename one measure both ways: once in the GUI, once in TMDL. Note which feels faster for one measure, which for twenty.
2. Add a `description:` to every measure in one table using TMDL alone.
3. Create a calculated column in TMDL (not the GUI). Confirm it appears in the field list.
4. Break the TMDL on purpose (misspell `measure` as `meausre`) and read the error. Learning what breaks is how you learn what works.

Common traps

- **Mixing tabs and spaces.** TMDL is indentation-sensitive. Pick one per project. VS Code's "Convert indentation to tabs" fixes this.
- **Single vs double quotes.** `'Date' [Date]` uses single quotes around the table name because of the space-or-reserved-word rule. `Sales` needs no quotes. Get this wrong and the error is cryptic.
- **Dollar signs in format strings.** Escape them: `"\$,##0"` . Without the backslash some tools treat `$` as a token start.
- **Forgetting to click Apply changes.** Edits in TMDL view are not live until you apply. The yellow bar at the top is easy to miss.

Further reading

- Microsoft Learn: *Tabular Model Definition Language (TMDL) overview*.
- The `pbi-tools` project on GitHub for scripted TMDL manipulation.
- `tabular-editor` docs — Tabular Editor's scripting API uses the same object model.

Next: [Module 2 — User-defined functions \(UDFs\)](#).

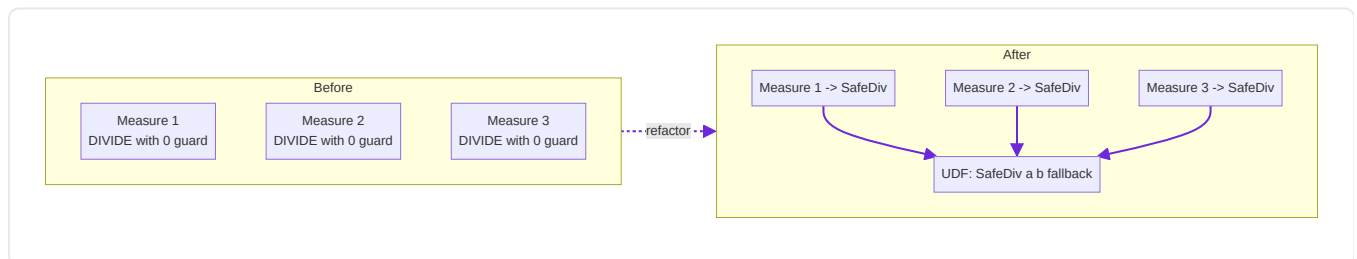
Module 2 — User-defined functions (UDFs)

A **DAX UDF** is a function you define once, with named parameters, and then call from any measure or calculated column. Think `def` in Python, or `CREATE FUNCTION` in SQL — finally arriving in DAX.

Why it matters now

Before UDFs, DAX had no first-class way to abstract a calculation. "Safe division", "same-period-last-year", "percent-of-total" lived as copy-pasted patterns across dozens of measures. One typo, 40 bugs. UDFs end that. They also matter for AI-authored measures: an agent that writes DAX against your UDF library produces consistent, reviewable code.

The mental model



One source of truth. Fix the UDF, fix every caller.

Hands-on walkthrough

1. Define a UDF in TMDL

In TMDL view, under the model root (not inside a table):

```
function 'SafeDiv' =
    (numerator: DOUBLE, denominator: DOUBLE, fallback: DOUBLE = 0) =>
        IF ( denominator = 0, fallback, DIVIDE ( numerator, denominator ) )
    description: "Division with a configurable fallback when the denominator is zero."
```

Points to notice:

- Parameters are **typed** (`DOUBLE`, `INT64`, `STRING`, `DATETIME`, `BOOLEAN`, or `TABLE`).
- `fallback: DOUBLE = 0` sets a default. Callers can omit it.
- The body is an expression after `=>`, same as a lambda.

2. Call the UDF from a measure

```
measure 'Gross Margin %' = SafeDiv ( [Gross Margin], [Total Sales] )
measure 'Return Rate %' = SafeDiv ( [Returns], [Total Sales], BLANK () )
```

Both measures use the same guard logic. The second overrides the default fallback.

3. A table-valued UDF

UDFs can return tables, which lets you abstract filter contexts:

```
function 'LastNDays' =  
  (n: INT64) =>  
    VAR Today = MAX ( 'Date'[Date] )  
    RETURN  
      FILTER ( ALL ( 'Date' ), 'Date'[Date] > Today - n && 'Date'[Date] <= Today )  
    description: "Returns the last N days ending at the latest date in context."
```

Call it inside `CALCULATE` :

```
measure 'Sales L7D' = CALCULATE ( [Total Sales], LastNDays ( 7 ) )  
measure 'Sales L30D' = CALCULATE ( [Total Sales], LastNDays ( 30 ) )
```

Compare to the pre-UDF version where each measure embeds the same `FILTER (ALL ... )` block.

A UDF library you can steal

```
function 'SafeDiv' =  
  (n: DOUBLE, d: DOUBLE, fallback: DOUBLE = 0) =>  
    IF ( d = 0, fallback, DIVIDE ( n, d ) )  
  
function 'PctOfTotal' =  
  (value: DOUBLE, total: DOUBLE) =>  
    SafeDiv ( value, total, 0 )  
  
function 'YoYDelta' =  
  (currentMeasure: DOUBLE, priorMeasure: DOUBLE) =>  
    currentMeasure - priorMeasure  
  
function 'YoYPct' =  
  (currentMeasure: DOUBLE, priorMeasure: DOUBLE) =>  
    SafeDiv ( currentMeasure - priorMeasure, priorMeasure )  
  
function 'FormatSignedPct' =  
  (x: DOUBLE) =>  
    IF ( x >= 0, "+" & FORMAT ( x, "0.0%" ), FORMAT ( x, "0.0%" ) )
```

Starting a new model? Paste this at the top of your TMDL and your measures get 30 % shorter.

When NOT to use a UDF

- Trivial one-liners (`SUM(Sales[Amount])`) — a UDF adds indirection with no benefit.
- Logic that depends on `SELECTEDVALUE` of a specific slicer — UDFs are pure functions of their parameters; smuggling filter context in through the back door makes them harder to reason about.
- As a replacement for calculation groups. Calc groups are for time-intelligence selectors; UDFs are for reusable expressions.

Exercises

1. Open an existing model with at least 15 measures. Count how many contain `IF ( ... = 0, 0, DIVIDE ... )`. Refactor them to `SafeDiv`.
2. Write a `PriorPeriod` UDF that takes a measure and a date column and returns the equivalent for the same period last year. Use it to collapse three measures into one.
3. Write a `MedianX` UDF (DAX has `MEDIAN` but not `MEDIANX` on an expression). Verify against a known dataset.
4. Deliberately pass the wrong type to a UDF. Note the error. UDFs give you type checking you didn't have before.

Common traps

- **Scope confusion.** UDFs live at the model level, not inside a table. Define them under `model` in TMDL.
- **Recursion is not supported.** A UDF cannot call itself (yet). Workaround: unroll a few levels, or use `GENERATESERIES` + `SUMX`.
- **Performance.** UDFs are compile-time inlined; a measure using a UDF is as fast as one with the body spelled out. But a UDF that wraps a slow expression is still slow — abstraction doesn't fix bad DAX.
- **Versioning.** Change a UDF's signature and every caller breaks. Treat UDFs like a public API: add a new one before deleting the old.

Further reading

- Microsoft Learn: *DAX user-defined functions*.
- SQLBI articles on DAX function design (predates UDFs but the design advice carries over).

Next: [Module 3 — PBIP + Git](#).

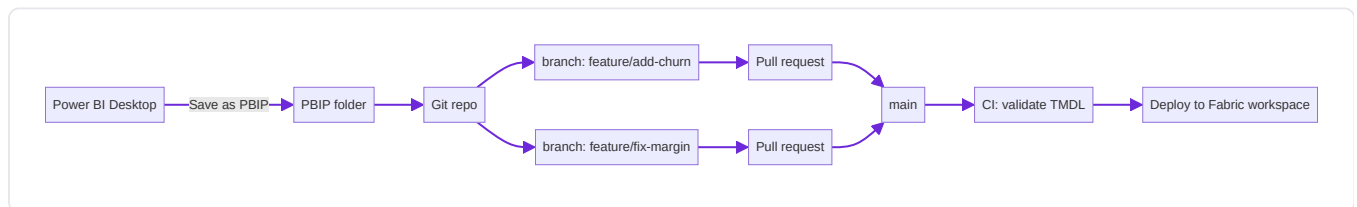
Module 3 — PBIP + Git

PBIP = Power BI Project format. Instead of saving as one binary `.pbix` file, you save as a folder of text files that Git can diff, review, branch and merge. Pair it with Git and you get the software-engineering practices Power BI was missing: pull requests, code review, CI/CD, rollback.

Why it matters now

Every "serious" data tool — dbt, SQL migrations, Airflow DAGs — treats code as the source of truth. Power BI stayed binary for a decade. PBIP ends that. For 2027 and beyond, assume every enterprise client wants their Power BI assets in Git.

The mental model



Same workflow a backend team uses, applied to a semantic model.

What a PBIP folder looks like

```
MyReport/
├── MyReport.Report/
│   ├── definition/
│   │   ├── pages/
│   │   │   ├── page1/
│   │   │   │   ├── page.json
│   │   │   │   └── visuals/
│   │   │   │       ├── visual-abc.json
│   │   │   │       └── visual-def.json
│   │   │   └── page2/...
│   │   └── report.json
│   └── definition.pbir
├── MyReport.SemanticModel/
│   ├── definition/
│   │   ├── tables/
│   │   │   ├── Sales.tmdl
│   │   │   └── Date.tmdl
│   │   ├── relationships.tmdl
│   │   ├── model.tmdl
│   │   └── database.tmdl
│   └── definition.pbism
├── .gitignore
└── MyReport.pbip <- small pointer file; open this in Desktop
```

Two artefacts, two folders: a Report and a SemanticModel. They can also live in separate repos if a central model serves many reports — the "thin report" pattern.

Hands-on walkthrough

1. Save a report as PBIP

Power BI Desktop → **File** → **Save as** → change filter to *Power BI project files (*.pbip)*. Pick a folder. Desktop writes the tree above.

2. Initialise Git

```
cd MyReport
git init
git add .
git commit -m "Initial PBIP checkpoint"
```

Use the Microsoft-provided `.gitignore` template for PBIP (search "PBIP gitignore" on Microsoft Learn). It excludes `.pbip/`, cache, and local user settings that don't belong in version control.

3. Make a change and see the diff

Add a measure in Desktop, save, then:

```
git diff
```

You will see a clean textual diff of `Sales.tmdl` :

```
+ measure 'Churn Rate %' =
+     SafeDiv (
+         [Churned Customers],
+         [Customer Count]
+     )
+     formatString: "0.0%"
+     displayFolder: "03 Customers"
```

Compare that to pre-PBIP, where "a measure changed" meant a 400 KB binary blob changed and nobody could tell what.

4. Branch, PR, review

```
git checkout -b feature/add-churn
# edit model in Desktop or TMDL
git commit -am "Add Churn Rate % measure"
git push -u origin feature/add-churn
```

Open a PR on GitHub / Azure DevOps / GitLab. Reviewers see the TMDL diff inline. Approve, merge, done. This is the exact workflow a Python team has used for 15 years.

5. Deploy from the PBIP

Three common paths:

- `main` is deployable; always matches the Dev or Test workspace.
- Prod deployment is a tag, not a branch. Rolling back = redeploy a previous tag.

Exercises

1. Convert an existing `.pbix` to PBIP. Commit. Make three small changes, each in its own commit. Read `git log --stat`.
2. Create a feature branch, add a measure, open a PR against yourself. Review your own diff out loud — would a reviewer understand?
3. Set up `.gitignore` from the Microsoft template. Confirm `cache/` and user state are excluded.
4. Add a GitHub Action that fails on any TMDL file over 500 lines (encourages splitting big tables).

Common traps

- **Merge conflicts in visual JSON.** Two people edit the same page → JSON conflict in `visual-xyz.json`. Resolvable but ugly. Solution: split work by page or by table.
- **Committing the `cache/` folder.** Blows up repo size. Always `.gitignore` it.
- **Opening the Report folder, not the `.pbip` pointer.** Desktop needs the `.pbip` file; the folders alone won't open directly.
- **Thin report + shared model drift.** If a thin report lives in repo A and its semantic model in repo B, pin the model version somewhere. Otherwise the report breaks silently when the model changes.
- **Secrets in `database.tmdl`.** Connection strings sometimes leak passwords. Parameterise connections; keep real credentials in the Service's data source settings.

Further reading

- Microsoft Learn: *Power BI Projects (PBIP) overview*.
- Microsoft Learn: *Fabric Git integration*.
- `pbi-tools` GitHub project.
- Rui Romano's blog — he drove much of PBIP's design.

Next: [Module 4 — MCP + AI agents](#).

Module 4 — MCP + AI agents

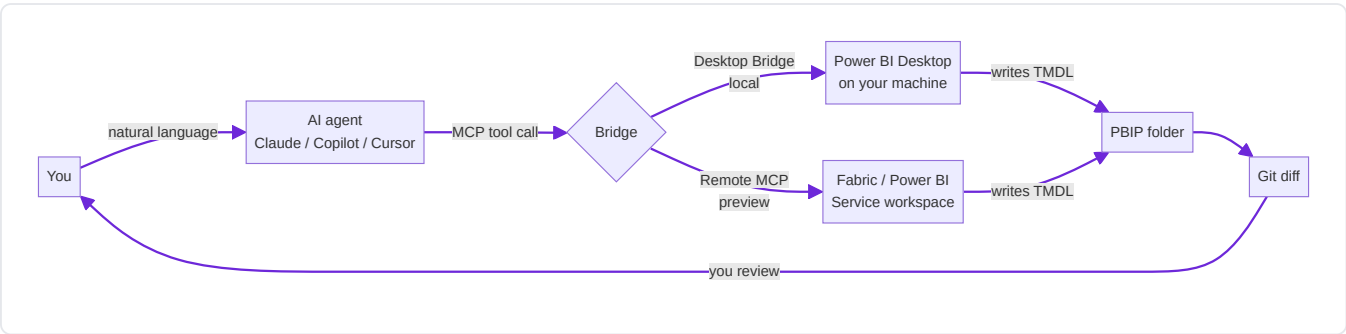
MCP = Model Context Protocol — an open standard that lets an AI client (Claude, Copilot, Cursor, ...) call tools on a server. Microsoft ships MCP servers for Power BI so an agent can read your semantic model, propose changes, generate DAX, write documentation, and apply edits — either locally through the **Desktop Bridge** or remotely through the **Remote Power BI Authoring MCP** (preview).

This is the final skill in the four-skill prep ladder. It's last for a reason: you need TMDL (to read what the agent writes), UDFs (to give the agent a clean vocabulary), and PBIP + Git (to roll back when the agent does something stupid). Without those three, agentic authoring is a footgun.

Why it matters now

The agentic workflow is where Power BI product investment is going. Microsoft calls it **Power BI Agentic Experiences** and it's GA. Developers who can supervise an agent end up 3–5× more productive on model work. Developers who can't, won't.

The mental model



Agent proposes, Git records, human approves. The agent is a very competent junior; you are still the senior reviewing the PR.

Two deployment modes

Mode	Where the model lives	Status	Use when
Desktop Bridge MCP	On your Windows machine, inside Desktop	GA	Local authoring; you have Desktop open anyway.
Remote Authoring MCP	In a Fabric / Power BI workspace	Preview	CI pipelines, cloud-native agents, headless automation.

Hands-on walkthrough — Desktop Bridge

1. Install prerequisites

- Current Power BI Desktop.
- An MCP-capable client. Example: Claude Code, Claude Desktop, Cursor, or Copilot in VS Code.

2. Configure your MCP client

Example `~/.claude/mcp.json` for Claude Code:

```
{
  "mcpServers": {
    "powerbi-desktop": {
      "command": "pbi-mcp",
      "args": ["--mode", "desktop-bridge"]
    }
  }
}
```

Equivalent for Cursor / VS Code uses their own `mcp.json`. Check the docs for your client — the exact file path moves around.

3. Open a PBIP in Desktop

The bridge needs a running Desktop instance with your PBIP open. Agents cannot (yet) spawn Desktop headlessly through the Bridge.

4. Ask the agent to do something

You: Read the Sales table and propose five KPI measures a sales manager would want. Add them to a new display folder called "01 Manager KPIs".

Agent: [calls MCP tool: get_table("Sales")]
[calls MCP tool: get_measures()]
[proposes 5 measures in TMDL]
[calls MCP tool: write_measures(...) after your approval]

5. Review the diff

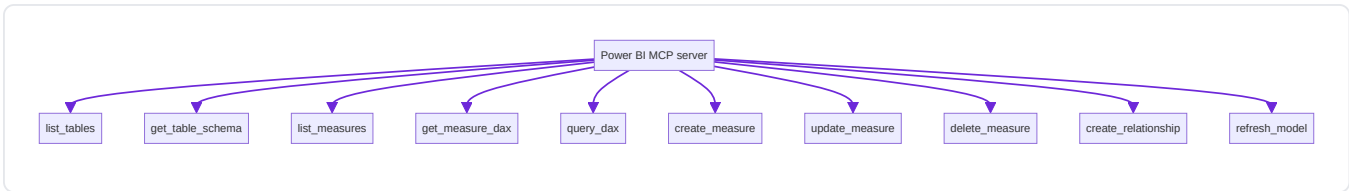
```
git diff
```

The agent's edits land in `Sales.tmdl` as a normal commit candidate. Read every line. If it added a `CALCULATETABLE` where `CALCULATE` would do, reject.

6. Commit or revert

```
git add . && git commit -m "Agent: 5 manager KPIs for Sales"
# or
git checkout . # throw it away
```

Tool surface typically exposed by the Power BI MCP server



Read tools are safe. Write tools should be gated by human approval in your client — Claude Code prompts; Cursor has a per-tool allowlist.

Patterns that work

- **Documentation generator.** "Read every measure in the model and add a `description:` property explaining what it computes in business terms." Review the TMDL diff; commit.
- **DAX refactor to UDFs.** "Find every measure that does `IF(x = 0, 0, DIVIDE(a, b))` and refactor to `SafeDiv(a, b)`."
- **Best-practice sweep.** "Run the Fabric BPA ruleset against this model and propose fixes for every High severity rule."
- **New model from a brief.** Hand the agent a one-page business brief and a star-schema SQL source. Get a first-draft TMDL. Expect to rewrite 40 % of it, but the 60 % you keep saves hours.

Patterns that don't work (yet)

- **"Build the whole report."** Agents are poor at report-layout decisions; the result looks like a 2015 dashboard.
- **"Optimise performance."** They will propose vaguely plausible rewrites that often make things slower. Use DAX Studio.
- **Multi-turn long edits without Git checkpoints.** The agent loses track. Commit frequently; a 10-minute agent session is 5 commits.

Remote Authoring MCP (preview)

Same tool surface, different endpoint:

```
{
  "mcpServers": {
    "powerbi-remote": {
      "command": "pbi-mcp",
      "args": [
        "--mode", "remote",
        "--workspace", "https://api.fabric.microsoft.com/v1/workspaces/<id>",
        "--auth", "service-principal"
      ]
    }
  }
}
```

The agent now operates on a model deployed to a workspace, no Desktop required. Useful for:

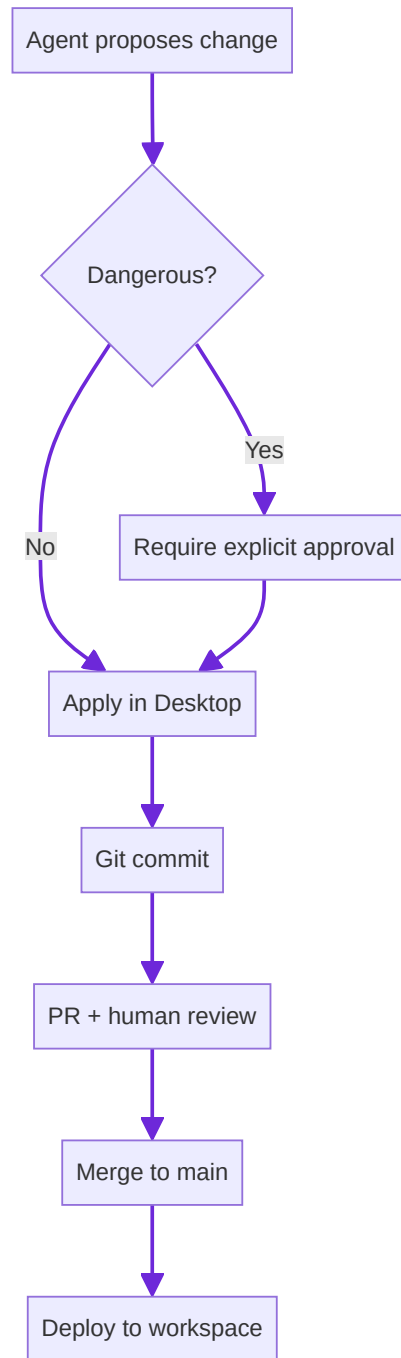
- Nightly CI: "diff the model against the BPA ruleset, open a GitHub issue if any High-severity rule fails."
- Chatbot that answers "add this measure for me" via a Teams front end.

Caveat: preview = breaking changes between monthly releases. Pin a specific MCP server version in production.

Exercises

1. Install an MCP client. Point it at the Desktop Bridge. Have it list the tables in any model you own.
2. Ask the agent to add a one-line `description:` to every measure in one table. Read every edit before accepting.
3. Ask the agent to generate a measure that does something wrong on purpose (e.g. divides without a zero guard). Reject it with a reason the agent can learn from.
4. Set up Git so you can `git revert` an agent commit without touching anything else.

Safety and governance



Minimum controls for a team setting:

- MCP write tools require per-call approval in the client.
- All agent output goes through PBIP + Git — no agent writes to the Service without PR review.
- Service principals used by remote MCP have **workspace-scoped** permissions, never tenant-wide.
- Log every tool call. Fabric audit logs cover remote MCP; capture Desktop Bridge calls with your MCP client's log.

Common traps

- **Letting the agent "just fix it" with write access, no review.** You will discover a broken production model on a Monday. Don't.
- **Giving the agent service-principal creds that can write to prod.** Give it dev only; promote through your deployment pipeline.
- **Hallucinated column names.** Agents sometimes invent columns. BPA rules + CI compile step catch this before deploy.
- **Prompt injection from the model's data.** Column `description` fields and table `description` fields are read by the agent as part of its context. If a user with write access to source metadata puts instructions there, the agent may follow them. Treat descriptions as untrusted input in multi-tenant settings.

Further reading

- Microsoft Learn: *Power BI Agentic Experiences* overview.
- Microsoft Learn: *Power BI MCP server* (Desktop Bridge).
- Microsoft Learn: *Remote Power BI Authoring MCP* (preview).
- Anthropic / MCP spec: modelcontextprotocol.io.

Next: [Module 5 — Copilot + Fabric IQ](#).

Module 5 — Copilot + Fabric IQ

Two overlapping surfaces, often confused:

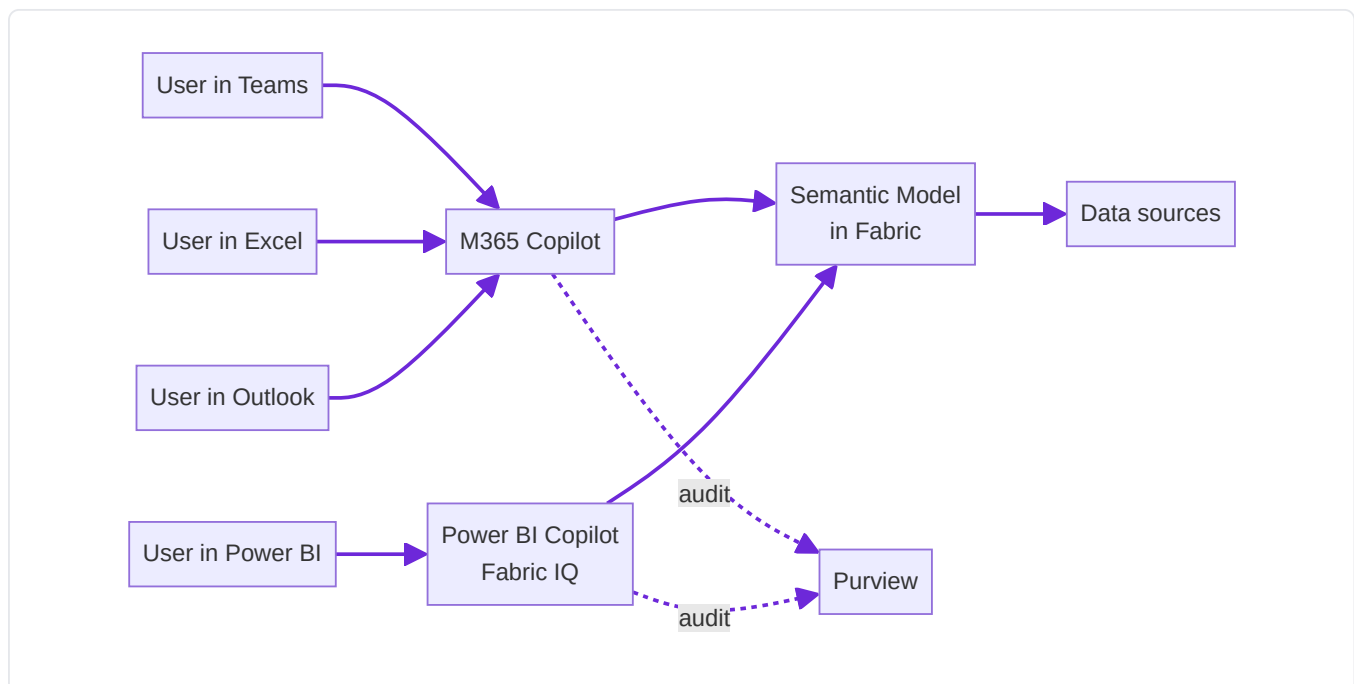
- **Microsoft 365 Copilot + Power BI** (preview) — the Copilot inside Word, Excel, Teams, Outlook can reach into trusted Power BI semantic models and answer business questions using them.
- **Fabric IQ + Power BI Copilot** (rolling GA) — Copilot inside the Power BI / Fabric experience itself. Reasons over a semantic model, generates visuals, explains results, shows the underlying DAX / PAX query.

Different entry points, same underlying idea: the semantic model is the trusted vocabulary a language model uses to answer business questions.

Why it matters now

Executives type questions into Microsoft 365 Copilot every day. Whether they get a useful answer depends on whether the semantic model behind Power BI is clean, documented, and labelled correctly. Your model is your product's AI interface, whether you planned for that or not.

The mental model



Both Copilots route through the semantic model. If the model's labels, descriptions and synonyms are weak, both Copilots answer badly.

What "trusted semantic model" means in Copilot terms

A semantic model Copilot can use well has:

- **Clear table and column names.** `fact_sales_daily` is machine-y; `Sales` reads naturally. Pick the one your business users would say.
- **Descriptions on every measure.** Copilot reads `description:` as prompt context. Empty descriptions = guesswork.
- **Synonyms configured.** "Revenue", "Sales", "Top line" should all resolve to `[Total Sales]`.
- **Row-level security (RLS) that holds.** Copilot answers through the calling user's identity. A broken RLS rule leaks data to Copilot too.
- **Verified answers** (Fabric IQ feature). Register a canonical question → canonical measure mapping. Copilot uses the mapping instead of guessing.

Hands-on walkthrough

1. Enable Copilot on a workspace

Fabric portal → **Admin portal** → **Tenant settings** → **Copilot** → turn on for your workspace. Requires a Fabric F64+ or Power BI Premium capacity (or P-SKU; or PPU for individual use).

2. Prepare one model for Copilot

In TMDL, add descriptions and synonyms:

```
table 'Sales'
  description: "One row per sales transaction. Grain: transaction line."

  measure 'Total Sales' = SUM ( Sales[Amount] )
    description: "Total sales amount in USD. Also known as revenue or top line."
    formatString: "\$#,##0"
    annotation PBI_FormatHint = "Currency"
    annotation Synonyms = "revenue, top line, gross sales"
```

The `Synonyms` annotation is the main hook for M365 Copilot's Q&A-style resolution.

3. Ask Copilot in the Power BI Service

Open the model → Copilot pane → type:

"What were last quarter's sales by region, with YoY growth?"

Copilot does three things visibly:

1. Generates a visual (likely a bar chart with a secondary metric).
2. Shows the DAX / PAX query it used.
3. Explains in natural language what the chart shows.

Step 2 is the one you care about as a developer. Read the DAX. If it chose a wrong measure or a wrong filter, the fix is in the model's descriptions and synonyms, not in a Copilot setting.

4. Ask via Microsoft 365 Copilot (preview)

From Teams Copilot chat with a semantic model grounded:

"/powerbi How are North-America sales tracking vs plan this month?"

Same reasoning path, different wrapper. Answer appears in the chat with a link back to the Power BI report.

Fabric IQ specifically

Fabric IQ is Microsoft's umbrella for the AI reasoning layer across Fabric (not just Power BI). For Power BI developers, three features matter today:

- **Verified Answers.** Register the canonical measure for a canonical question. Copilot answers from the mapping before trying to generate DAX. Think "FAQ" for your semantic model.
- **Semantic model explanations.** Copilot can explain any measure in natural language on hover, using `description:` + the DAX body.
- **PAX visibility.** The query Copilot generates is shown and copy-pastable, which makes it debuggable like any other DAX.

A review checklist before you let Copilot near a model

- ☐ Every measure has a `description:`.
- ☐ Table and column names are user-friendly (not warehouse names).
- ☐ Synonyms added for the top 20 measures.
- ☐ RLS tested with at least two personas; verify Copilot respects it.
- ☐ Date table marked as a date table; `[Date]` is the key column.
- ☐ No columns named `Measure1`, `Column2`.
- ☐ Verified Answers registered for the 5 most-asked questions.
- ☐ Sensitive measures (margin, salary, PII counts) are either excluded from Copilot or gated by RLS.

Skip this list and Copilot's answers will embarrass you.

Live visuals in Outlook (preview)

Not strictly a Copilot feature but adjacent: users can embed a Power BI visual in Outlook on the web and interact with it inside the email — filter, cross-highlight, drill. Governance notes:

- Same RLS, same workspace permissions. Users without access see nothing.
- Rendering uses the recipient's identity, not the sender's.
- Don't send to external recipients unless the workspace supports guest access.

Exercises

1. Pick a model you own. Count how many measures have an empty `description`. Write descriptions for the top 10 measures.

2. Add synonyms for your top 5 measures. Ask Copilot a question using each synonym. Verify it resolves correctly.
3. Deliberately give one measure a misleading description. Ask Copilot a question that would use it. Notice how the wrong description → wrong answer. Now you understand why descriptions are code.
4. Register a Verified Answer for one canonical question. Ask the question two different ways. Confirm Copilot uses the mapping.

Common traps

- **"Copilot is hallucinating."** Nine times out of ten, the model is under-documented. The fix is in TMDL, not in Copilot settings.
- **Capacity-sized bills.** Copilot queries consume Fabric capacity units. A chatty executive user can blow a daily budget. Monitor.
- **Thin reports with shared models.** Copilot answers using the model's metadata; a thin report's layout doesn't help. Invest in the model.
- **Preview behaviour changes.** Microsoft tunes Copilot's prompts monthly. An answer that worked last week may differ this week. Verified Answers insulate you.

Further reading

- Microsoft Learn: *Copilot for Power BI overview*.
- Microsoft Learn: *Fabric IQ*.
- Microsoft Learn: *Prepare your data for Copilot in Power BI*.

Next: [Module 6 — Service features and UX updates](#).

Module 6 — Service, DirectQuery and UX updates

Grab-bag of features that aren't the headline agentic story but matter for day-to-day work. All GA except where noted.

1. DirectQuery semantic models in the Service
2. Visual Calculations
3. Initial Axis Scroll Position
4. New Design Ribbon
5. Live Power BI visuals in Outlook (preview — see Module 5)

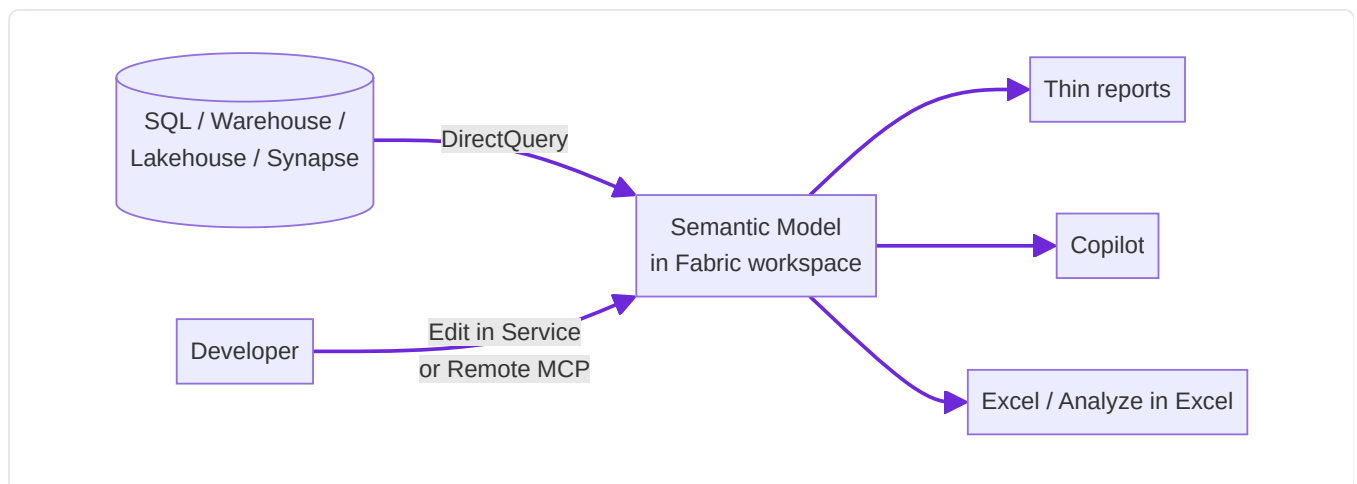
1. DirectQuery semantic models in the Service

You can now **create and edit** DirectQuery semantic models directly in the Power BI / Fabric Service — no Desktop round-trip. The Service authoring experience caught up to Desktop for most model edits.

Why it matters

- Cloud-first teams can build and iterate without a Windows machine.
- CI/CD pipelines can patch a live model via Fabric REST without publishing a `.pbix`.
- Pairs with Remote MCP (Module 4): an agent can edit a DirectQuery model running in production.

Mental model



Model stays in the Service. Queries hit the source live. Edits happen in the browser or via API.

When to use DirectQuery vs Import

Use DirectQuery when	Use Import when
Source is already fast (Fabric Warehouse, Lakehouse SQL endpoint, Synapse)	Source is slow or external
You need real-time or near-real-time data	Daily / hourly refresh is fine
Data is too large to fit in Premium capacity memory	Data comfortably fits
RLS must live in the source for compliance	RLS in the model is OK

Composite models (part import, part DQ) are usually the right answer for anything non-trivial.

Hands-on

1. In a Fabric workspace → **+ New → Semantic model → DirectQuery**.
2. Pick a source (a Fabric Lakehouse works for a quick test).
3. Drag tables in; set relationships.
4. **Open data model in Service** → edit measures, relationships, descriptions — all in the browser.
5. Build a thin report that connects to this model by *Live connection*. Keep report and model in separate PBIP folders.

Traps

- **Latency.** Every slicer click fires queries. Measure query times in the Performance Analyzer before shipping.
- **Measure complexity.** Some DAX patterns (iterated `SUMX` over big tables) collapse to monster SQL. Use DAX Studio to inspect generated SQL.
- **No calculated columns on DQ tables** unless the source supports them. Push the calc to the source instead.

2. Visual Calculations (GA)

A calculation that lives **on the visual**, not in the semantic model. It sees the visual's rows and columns the way a human reads them.

Why it matters

Before Visual Calculations, "moving average over the current chart's timeline" meant creating a model-level measure, polluting the model with report-specific logic. Now the calc lives on the visual and disappears when the visual does.

Syntax quick tour

Inside the visual's **Add visual calculation** pane:

```

-- 3-period moving average of Total Sales across the visual's rows
MovingAverage = MOVINGAVERAGE([Total Sales], 3)

-- Running total down the visual
RunningSales = RUNNINGSUM([Total Sales])

-- Previous row's value
Prev = PREVIOUS([Total Sales])

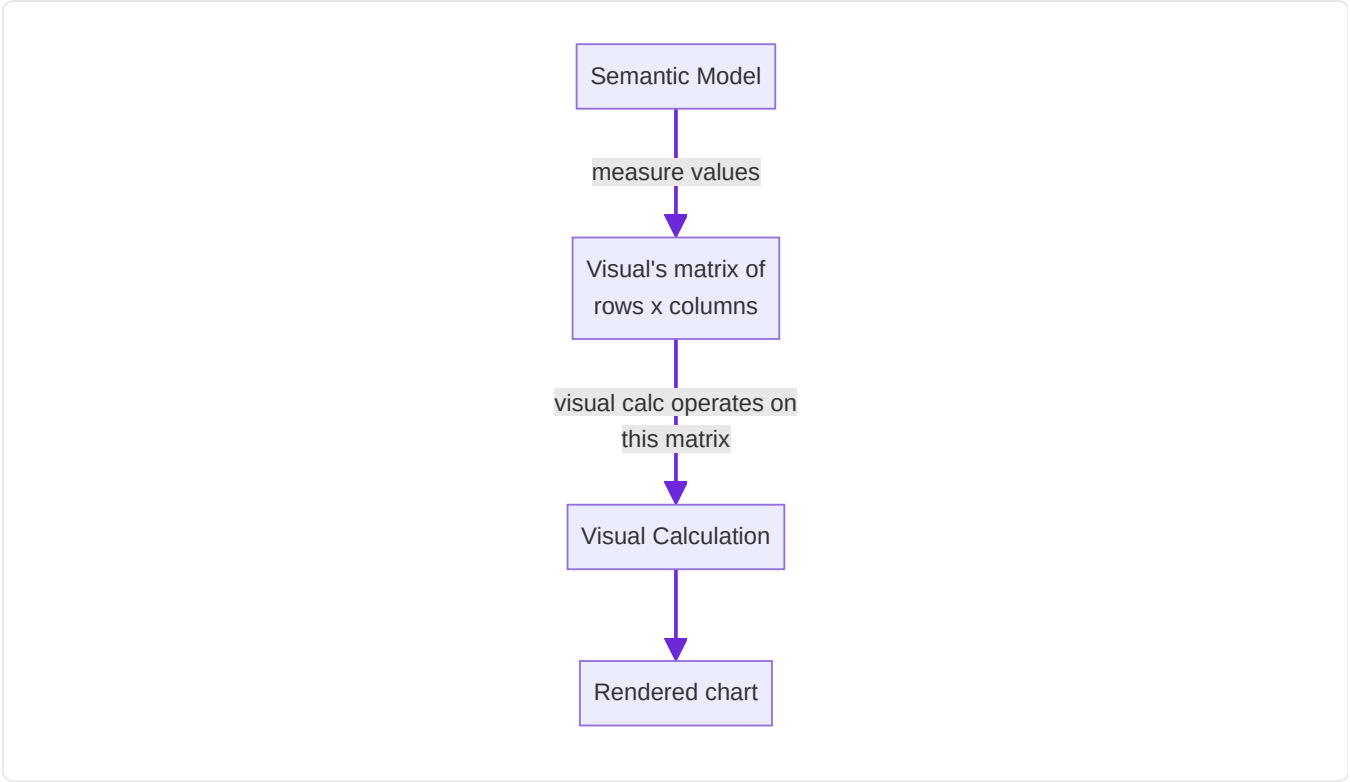
-- Percentage change vs previous row
PctChange = DIVIDE([Total Sales] - PREVIOUS([Total Sales]), PREVIOUS([Total Sales]))

-- Index of the current row
Rank = RANKX(ALLSELECTED(ROWS), [Total Sales])

```

Available functions: PREVIOUS , NEXT , FIRST , LAST , MOVINGAVERAGE , RUNNINGSUM , RANKX , COLLAPSE , EXPAND , COLLAPSEALL , EXPANDALL , plus full DAX.

Mental model



Think "spreadsheet formula on the visual's grid". PREVIOUS means "the cell above", not "a time-intelligence trick".

When to prefer Visual Calcs over model measures

Prefer visual calc	Prefer model measure
Running totals, moving averages for one chart	Any value used in multiple reports
Row-over-row comparisons	Cross-report consistency
Rank within the visible slice	Values exposed to Copilot
Quick prototypes	Values with business-critical definitions

Trap: Copilot doesn't see visual calcs

Copilot reasons over the semantic model only. If a KPI matters to executives via Copilot, make it a model measure (with description + synonyms), not a visual calc.

3. Initial Axis Scroll Position (GA)

A category/time axis can be configured to open at the **latest** value instead of the earliest. For an operational dashboard looking at the last 24 hours, this is table stakes.

How

On the X-axis properties for a chart, enable **Initial scroll position → End**. Combine with a scrollable axis and a sensible window.

Why it matters

A manager opening "last 90 days of orders" wants to see today, not Day 1. Pre-feature workaround was a **TOPN** filter that broke drill-down.

4. New Design Ribbon (GA)

Centralised control for themes, page size, background, wallpaper and report styling. Not a technical feature — a UX one — but it's where every report-styling task lives now.

What moved there

- Theme selection and customisation (previously under View → Themes).
- Page size and orientation (previously under Format pane).
- Background / wallpaper images.
- Report-wide filter styling.

Why it matters for your workflow

Reports that look professional in 2026 use custom themes shipped as **.json**. Keep theme JSON in the PBIP repo:

```
MyReport/  
├─ themes/  
│   └─ brand-light.json  
│   └─ brand-dark.json
```

Reference them from Design ribbon → Import theme. Version-controlled themes = consistent look across every report in a tenant.

5. Live Power BI visuals in Outlook (preview)

Covered in Module 5. Short version: embed a visual in an email, the recipient interacts with it live in Outlook on the web, RLS applies.

Exercises

1. Build a DirectQuery model against a Fabric Lakehouse. Edit a measure entirely in the Service (no Desktop). Confirm the edit via a thin report.
2. Add a Visual Calculation to a line chart: running total + 7-point moving average. Note what works that would have needed a model measure before.
3. Open an operational dashboard chart. Flip Initial Axis Scroll to End.
4. Export your tenant's default theme as JSON, commit to Git, modify two colours, re-import. You now have version-controlled branding.

Common traps across this module

- **Mixing authoring surfaces.** Edit the same model in Desktop and in the Service in parallel → conflicts. Pick one authoring surface per change; use PBIP + Git to coordinate.
- **DirectQuery with no capacity planning.** DQ workloads spike CU usage. Monitor Fabric capacity metrics before promoting to prod.
- **Over-using visual calcs.** Report-level logic sprawls fast. If a calc appears on three visuals, it belongs in the model.

Further reading

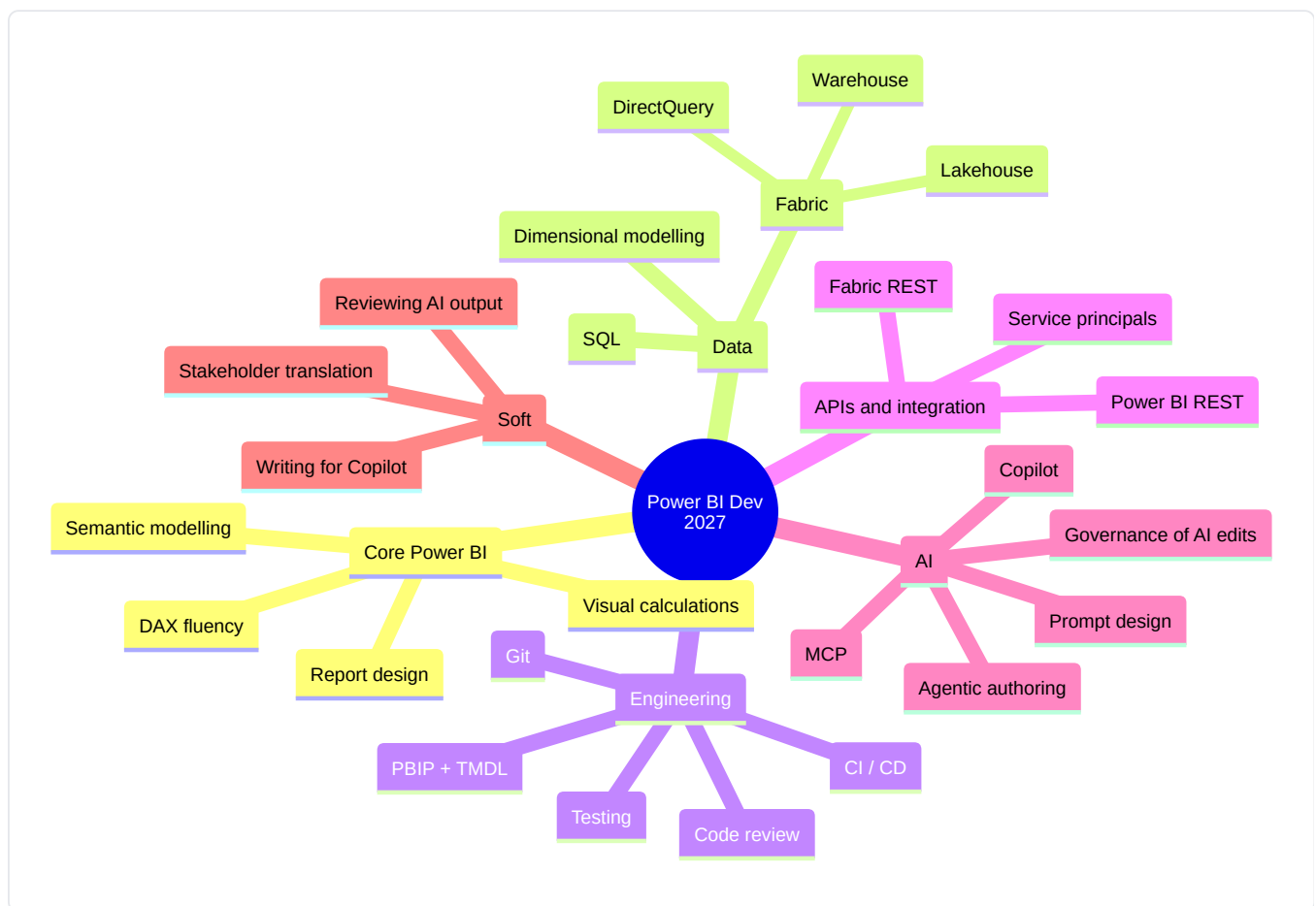
- Microsoft Learn: *Create semantic models in the Service*.
- Microsoft Learn: *Visual calculations overview*.
- Power BI monthly release notes (search "Design ribbon", "Visual calculations", "Initial scroll").

Next: [Module 7 — Roadmap and skill stack](#).

Module 7 — The 2027 skill stack and a 90-day roadmap

The thesis of this course in one sentence: **Power BI development is becoming analytics engineering with an AI pair.** The job description that paid in 2022 does not pay in 2027.

The 2027 skill stack



If you are strong in two branches and weak in four, you are a 2022 developer. Aim for competence in all six by end of 2026.

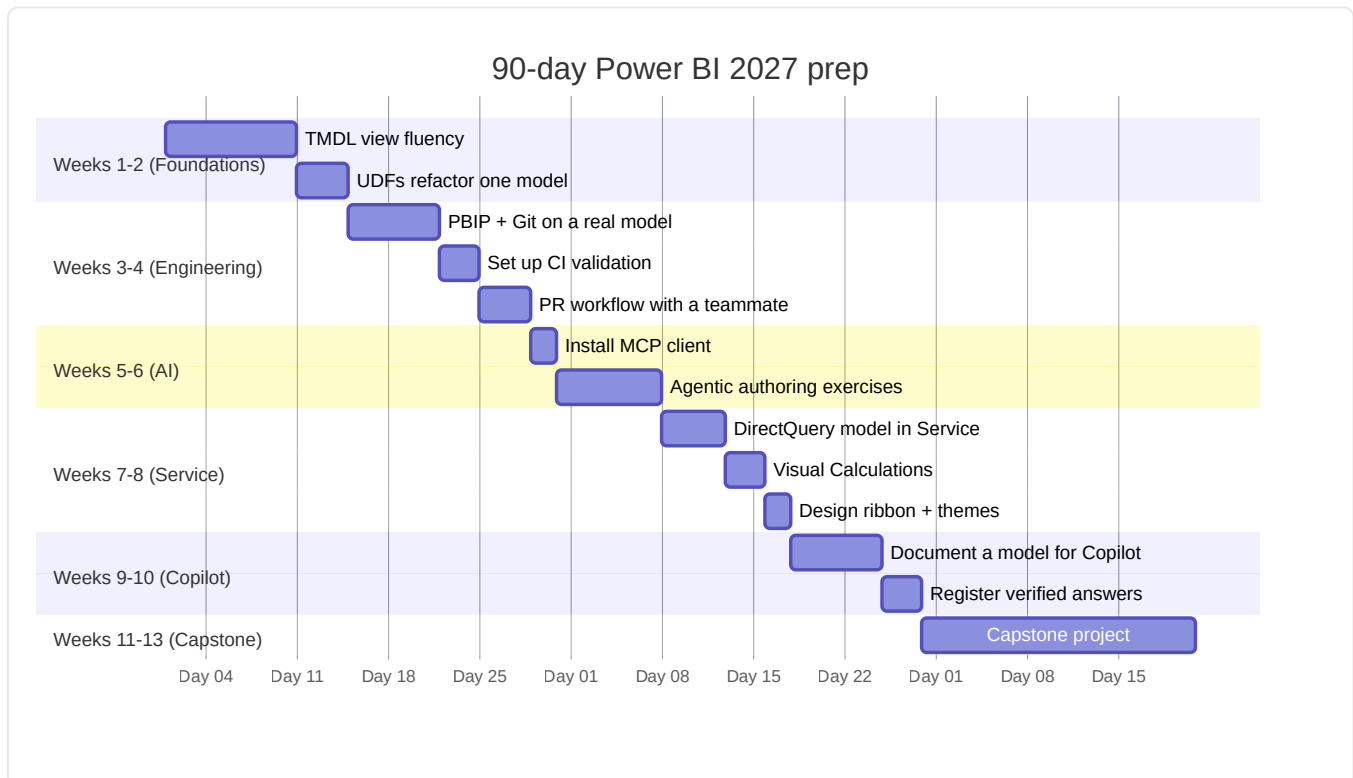
What's dropping out

- Expertise in `.pbix` as the unit of work. PBIP replaces it.
- Clicking through the UI as the primary authoring mode. Still useful; no longer sufficient.
- "I don't do code reviews." The industry does.
- Doing all the DAX writing yourself. You review DAX now. The agent drafts.

What's rising

- **Semantic model as product.** The model is an API consumed by humans, Copilots, agents, and downstream reports. Treat it like one.
- **Writing for Copilot.** Descriptions, synonyms, verified answers. Non-trivial craft.
- **Reviewing AI edits.** The highest-leverage skill of the next three years. If you can tell "subtly wrong DAX" from "correct DAX", you're valuable.
- **Governance.** Who can let the agent write to prod? Auditable how? Someone has to answer this. Might as well be you.

90-day roadmap



Capstone project

Pick a dataset you care about (public or your own). Deliver:

1. A PBIP in Git with meaningful commits and at least one merged PR.
2. A semantic model with:
 - TMDL you wrote by hand for at least one table.
 - A UDF library of ≥ 3 reusable functions.
 - Descriptions and synonyms on every measure.
 - RLS with two personas.
3. A thin report that uses Visual Calculations where appropriate.
4. One Copilot interaction documented (prompt → DAX → explanation), with a short note on how the model's documentation helped.
5. One agent interaction (MCP) that proposes a change, which you review and either merge or reject with written reasoning.
6. A CI job that validates TMDL on every PR.

Portfolio piece. Push it to GitHub. Link from your CV.

| How to keep learning after the 90 days

- **Power BI monthly release notes.** Read every one. 15 minutes.
- **Microsoft Fabric blog.** The headline announcements live there.
- **Rui Romano, Chris Webb, Marco Russo / Alberto Ferrari, Reid Havens, Guy in a Cube.** Four blogs and one YouTube channel will keep you current.
- **MCP spec repo on GitHub.** Watch releases.
- **One side project per quarter.** Reading isn't learning.

| Reality check, closing

A Power BI developer who ignores this stack through 2026 will spend 2027 competing with developers who didn't. The tools are not magic; they are learnable in a quarter of focused evenings. The hard part is noticing in time that the ground shifted. You just did. Start Module 1.

[← Back to README](#)